

NumPy and Numerical Computing: Worked Solutions

Problem 1. Shapes and reductions

Problem. For X of shape $(7, 4)$, give the shapes of $X.\text{mean}(0)$, $X.\text{mean}(1, \text{keepdims}=\text{True})$, $X[:, \text{None}, :]$, and $X.T @ X$. Explain every remaining axis.

Solution. Interpret rows as observations and columns as features. A reduction combines entries along the named axis; that axis disappears unless `keepdims=True` retains it with length one.

Expression	Shape	Meaning of the remaining axes
<code>X.mean(0)</code>	$(4,)$	one mean per feature
<code>X.mean(1, keepdims=True)</code>	$(7, 1)$	observation; reduced feature axis
<code>X[:, None, :]</code>	$(7, 1, 4)$	observation; inserted singleton; feature
<code>X.T @ X</code>	$(4, 4)$	first feature; second feature

The last expression contracts the observation index:

$$(X^T X)_{jk} = \sum_{i=1}^7 X_{ij} X_{ik}.$$

It is an uncentered Gram matrix, not generally a covariance matrix. The inserted singleton in the third expression performs no reduction and introduces no new observations. A $(4,)$ array has one axis; its transpose has the same shape.

Problem 2. Pairwise distances with bounded temporary storage

Problem. For row matrices $X \in \mathbb{R}^{n \times d}$ and $Y \in \mathbb{R}^{m \times d}$, implement $Q_{ij} = \|x_i - y_j\|_2^2$ using at most b rows of X at a time. Determine the peak temporary storage and compare with direct differences, including singleton axes.

Solution. A block with $r \leq b$ rows creates differences of shape (r, m, d) . Contracting the feature axis gives an (r, m) block. The following implementation assumes float64 input arrays and $b \geq 1$.

```
import numpy as np

def blocked_distances(X, Y, b):
    Q = np.empty((len(X), len(Y)), dtype=np.float64)
    for start in range(0, len(X), b):
        stop = min(start + b, len(X))
        delta = X[start:stop, None, :] - Y[None, :, :]
        Q[start:stop] = np.einsum('ijk,ijk->ij', delta, delta)
        del delta
    return Q

rng = np.random.default_rng(17)
for n, m, d in [(7, 4, 3), (1, 4, 3), (7, 1, 1)]:
    X = rng.normal(size=(n, d))
    Y = rng.normal(size=(m, d))
    direct = ((X[:, None, :] - Y[None, :, :])**2).sum(-1)
    for b in [1, 3, 20]:
        np.testing.assert_allclose(blocked_distances(X, Y, b),
                                    direct, rtol=1e-12, atol=1e-12)
```

Beyond the inputs and stored output, the principal temporaries contain at most $bmd + bm$ float64 entries, or $8(bmd + bm)$ bytes, with b replaced by $\min(b, n)$. Array metadata and small implementation buffers are excluded. The explicit deletion prevents an old difference block from surviving while the next is allocated. The total storage also includes the nm output entries; blocking does not remove that cost. Arithmetic remains $O(nmd)$.

The equivalent identity

$$Q_{ij} = \|x_i\|^2 + \|y_j\|^2 - 2x_i^T y_j$$

avoids the three-dimensional difference array, but subtracts large nearly equal numbers for nearby, large points. For $x = (10^8 + 1, 10^8 + 2)$ and $y = (10^8, 10^8)$, the exact squared distance is 5. Direct float64 differences recover it; the Gram expression can return 0 after rounding. Execution speed and accuracy of small distances are separate properties.

Problem 3. Cancellation and rounded inputs

Problem. Choose large-offset data with small deviations. Compare $\overline{x^2} - \bar{x}^2$ and $\overline{(x - \bar{x})^2}$ in float64 and float32. Separate formula error from rounding when the inputs are cast.

Solution. Set $x_i = c + e_i$ with $e = (-2, -1, 0, 1, 2)$. The exact mean is c and the population variance is

$$V = \frac{4 + 1 + 0 + 1 + 4}{5} = 2.$$

At $c = 10^4$, both dtypes represent all five inputs exactly. Nevertheless, the uncentered float32 calculation loses the variance by subtracting two quantities near 10^8 . At $c = 10^8$, float32 rounds all five inputs to the same value, so even a centered calculation receives constant data.

```
for offset in [1e4, 1e8]:
    exact_inputs = offset + np.arange(-2., 3.)
    for dtype in [np.float64, np.float32]:
        z = exact_inputs.astype(dtype)
        mu = z.mean()
        naive = (z*z).mean() - mu*mu
        centered = ((z-mu)**2).mean()
        represented = np.var(z.astype(np.float64))
        print(offset, dtype.__name__, naive, centered, represented)
```

Here **represented** measures the variance of the already rounded inputs with float64 arithmetic. Keeping **mu*mu** in the input dtype makes the comparison explicit.

c	dtype	Uncentered	Centered	Rounded-input variance
10^4	float64	2	2	2
10^4	float32	0	2	2
10^8	float64	0	2	2
10^8	float32	0	0	0

These are the outputs for the stated NumPy operations in the tested environment; alternative reduction implementations may round differently. The distinction is general: centering addresses cancellation during calculation, whereas promoting an already quantized input cannot restore lost deviations. Neither failure here is overflow: every intermediate remains finite.

Problem 4. A positive semidefinite sample covariance

Problem. Compute the sample covariance of random rows. Check symmetry and $v^T S v \geq 0$ for several vectors v , and explain why the inequality holds before rounding.

Solution. For $n \geq 2$, write $X_c = X - \mathbf{1}_n \bar{x}^T$, where $\bar{x}$ is the feature-mean vector. Then

$$S = \frac{X_c^T X_c}{n-1}, \quad S^T = S, \quad v^T S v = \frac{\|X_c v\|_2^2}{n-1} \geq 0.$$

Thus symmetry and positive semidefiniteness follow from the centered Gram representation. Positive definiteness additionally requires X_c to have full column rank; in particular, its rank cannot exceed $n - 1$.

```
rng = np.random.default_rng(17)
X = rng.normal(size=(200, 3))
C = X - X.mean(axis=0, keepdims=True)
S = C.T @ C / (len(X)-1)
np.testing.assert_allclose(S, np.cov(X, rowvar=False))
np.testing.assert_allclose(S, S.T, atol=1e-12)
for v in rng.normal(size=(10, 3)):
    lhs = v @ S @ v
    rhs = np.sum((C @ v)**2) / (len(X)-1)
    np.testing.assert_allclose(lhs, rhs, rtol=1e-12, atol=1e-12)
    assert lhs >= -1e-12
```

The tolerance permits tiny floating-point discrepancies on these small, well-scaled inputs. Checking ten directions alone would not prove positive semidefiniteness; the squared-norm identity proves it for every direction. The denominator $n - 1$ makes S unbiased for the population covariance under iid sampling with finite second moments.